

Coding & Solution

Date	1 July 2026
Team ID	xxxxxx
Project Name	Personalized Networking Assistant
Maximum Marks	5 Marks

Coding & Solution

This section evaluates the quality and completeness of your implemented code and the overall solution. Your submission should demonstrate working functionality, clean code practices, and alignment with your defined requirements.

Instructions:

1. Ensure your code is well-structured, commented, and follows best practices for your chosen technology stack.
2. The solution should address the core problem statement and implement the key functional requirements.
3. Include all source files, configuration files, and setup instructions needed to run the application.
4. Attach or link to your code repository (e.g., GitHub) in the table below.

Solution Summary

Field	Details
Repository Link / URL	Github-Personalized Network Assisstant
Programming Language(s)	Python
Framework(s) Used	FastAPI, Streamlit
Key Features Implemented	Event analysis, topic detection, personalized conversation generation, fact verification using Wikipedia API, conversation history tracking, feedback collection, PDF report generation, Swagger API documentation, unit testing
Pending / Incomplete Features	User authentication, database integration, cloud deployment, Gemini API integration
Setup / Run Instructions	1. Clone repository. 2. Create virtual environment. 3. Install dependencies using <code>pip install -r requirements.txt</code> . 4. Run backend using <code>python -m uvicorn app.main:app --reload</code> . 5. Run frontend using <code>streamlit run frontend/streamlit app.py</code> .

Code Quality Checklist

S.No	Criteria	Status (Yes / No)
1	Code is modular and organized into functions / classes	Yes
2	Meaningful variable and function names are used	Yes
3	Code includes comments / documentation where necessary	Yes
4	Error handling is implemented for critical operations	Yes
5	The application runs without critical errors	Yes
6	Code is committed to a version control repository	Yes

Additional Notes / Comments:

The project follows a modular architecture with separate frontend, backend, service, and testing layers. The application was successfully tested using PyTest, and all test cases passed successfully (5/5 tests passed). The project is fully functional and ready for further enhancements and deployment.